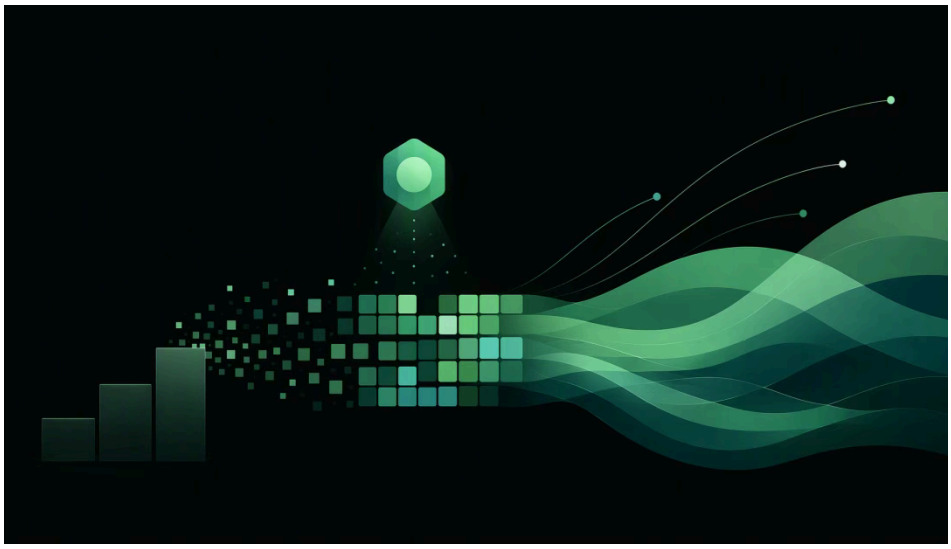
从榜单走向真实业务负载：Kernel Agent 刷新 Qwen3.8-Max 核心推理算子性能

原创 范睿博 阿里技术 2026年9月28日 03:54 浙江 4人



这是2026年的第 64 篇文章

（ 本文阅读时间：约 15 分钟 ）

在 60 个真实 shape 上，AKA 三条关键算子线的平均时延均低于专家版本；较生产/官方基线最高加速 1.55×。

关键词：GPU Kernel、Kernel Agent、FlashAttention-4、Gated DeltaNet、生产级优化

01
从 benchmark 到生产：验证标准正在改变

过去一年，Kernel Agent 已成为 GPU 软件栈的研究热点。EvoKernel、CUDA-Agent、NVIDIA AVO、KDA、CAKE 等工作，展示了 Agent 生成 Kernel、调用编译器与 profiler，并通过长周期迭代探索硬件性能上限的能力。

这些能力主要在公开 benchmark 上得到验证。KernelBench、SOL-ExecBench 等基准提供统一、可复现的测试场景；Qwen3.8 也曾在 SOL-ExecBench 的 FlashInfer-Bench 子集登顶。公开榜单证明了 Agent 能在给定题目上生成正确且高性能的 Kernel，但生产交付要求回答另一个问题：它能否直接替换现有的推理框架已经集成的高性能实现，并获得更多的性能收益？

两者的区别不只是输入规模更大。Benchmark 通常围绕有限的 shape、数据类型和调用接口；生产算子则必须同时覆盖真实shape 分布、混合精度的数据语义和 MTP 负载，并遵守算子在推理模型结构上下游中既有的输入/输出协议。优化不能只对少数 case 有效，也不能依赖特定输入分布或绕过必要计算；它需要在整组 workload 上保持精度、稳定性和总体收益，任一方面失效都可能使其无法上线。

因此，从榜单成绩走向生产，不只是继续压低一次 benchmark 的耗时，而是要在完整软件契约和工作负载约束下找到可以稳定部署的新执行路径。这也意味着，面向生产的 Kernel Agent 必须同时具备结构搜索能力和工程验证闭环。Atrex Kernel Agent（AKA）正是面向这一目标构建的生产级异构推理算子生成Agent，由阿里巴巴 TRE AI 系统工程团队研发，通过编码 Agent、性能 Profiling、GPU 评测与正确性门禁形成持续优化闭环。

Qwen3.8-Max 真实业务负载上的优化结果

作为面向大规模在线服务的旗舰模型，Qwen3.8-Max 每日承载数以亿计的调用，注意力和线性注意力算子位于每次推理的关键路径，单个 Kernel 的微秒级下降会在高并发流量中放大为吞吐提升、成本下降和服务余量。此次，AKA 将验证对象从公开 benchmark 推进到真实生产负载，并以生产/官方实现和专家手工优化版本为参照；在 FA4（FlashAttention-4）prefill、FA4 decode 和 GDN（Gated DeltaNet）prefill 三个关键算子、60 个代表业务负载的真实 shape 上，AKA 的几何平均时延全部低于专家版本，相关 Kernel 已经开源于atrex-kernels算子库 | https://github.com/alibaba/atrex-kernels。

三条算子线的几何平均时延均呈现同一排序：**AKA 最快，专家其次，生产/官方基线最后。**

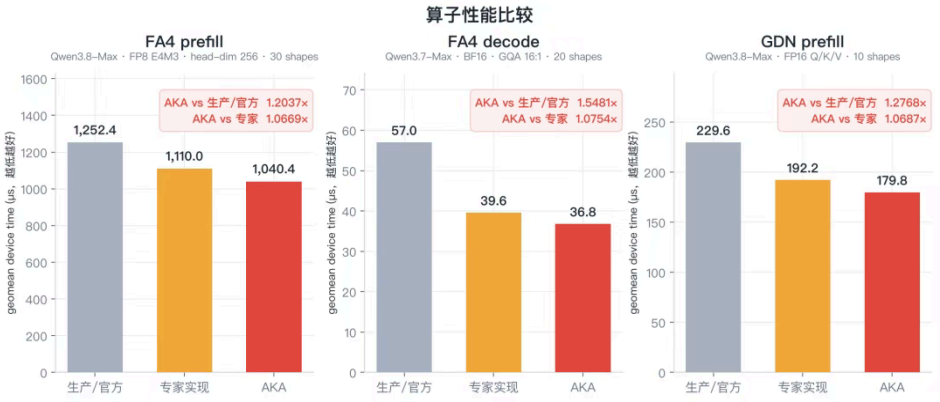


图 1 | 三条算子线完整 shape 集合的几何平均时延，数值越低越好。

加速比均为“基线时延 / AKA 时延”。

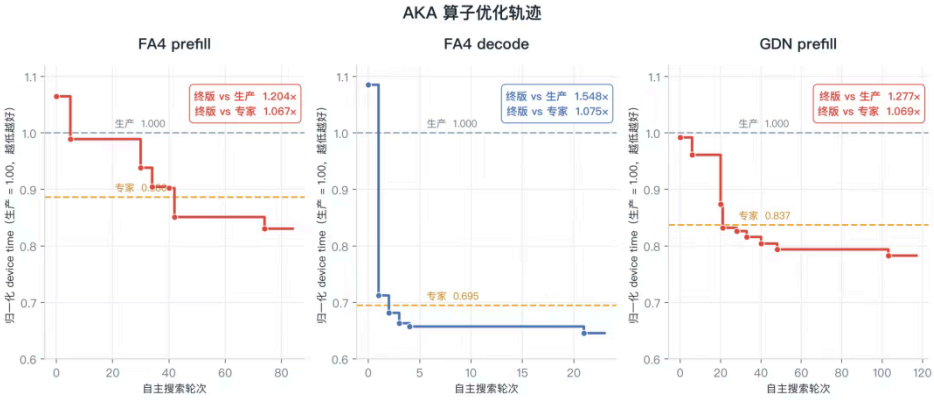


图 2 | FA4 prefill、FA4 decode 和 GDN prefill 的最优搜索轨迹。

生产实现归一化为 1.00，虚线分别表示生产和专家水平；

02
三类实现：生产实现、专家版本与公开基线

三条算子线均在同一硬件、同一接口、同一组 shape 上比较生产/官方实现、专家版本和 AKA。候选先通过数值正确性门禁，再进入完整 workload 的性能复测；AKA 以现有最优开源实现为起点进行增量优化。

- **FA4 prefill** 对比 FlashInfer–TRTLLM 生产路径（闭源）和 Atrex 专家手工实现；
- **FA4 decode** 对比公开 FlashInfer 和针对 Qwen decode 负载调优的 Atrex 专家实现；
- **GDN prefill** 对比 FlashInfer 官方 chunked 主干和独立的 Atrex M64 专家实现。

按时延严格比较，AKA 相对生产/官方基线分别有 30/30、19/20 和 10/10 个 shape 更快；相对专家则分别有 25/30、12/20 和 10/10 个 shape 更快。decode 的决定性增益集中在 q4 MTP。

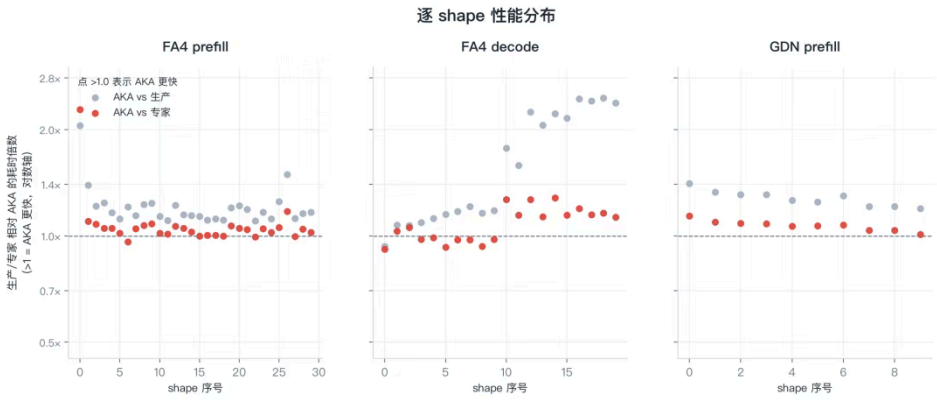


图 3 | 逐 shape 的基线/AKA 加速比分布。横线为 1.0× 持平线；点位高于 1 表示 AKA 更快。

03
三个案例：从结构重构到编译期特化

3.1 FA4 prefill：在常规单序列上重构持久化调度

在 batch=1、Q=KV=4224 的常规单序列 self-prefill 上，AKA 的时延为 63.42 μs，相对专家的 74.37 μs 加速 1.17×，相对生产的 94.60 μs 加速 1.49×。

这个 shape 使用 page64、16 个 Q head、1 个 KV head 和 256 维 head。AKA 与专家都使用单个 2-CTA Kernel，因此性能差异不来自多 Kernel 切分，而来自对调度和内部流水的进一步重构。

专家版本采用固定的 2-CTA tile 调度。AKA 则为 batch=1 的大 prefill 引入 persistent grid：常驻 cluster 跨约 4 个 wave 连续处理多个逻辑 tile，复用 pipeline、TMEM 和 barrier 状态，将启动、分配、同步与退出开销摊到更多工作上。单请求没有跨请求的负载不均，M-raster 还会将 causal attention 由轻到重的计算量交错分配给常驻 cluster。

数据通路也同步特化：原生 page64 TMA 直接按页搬运 K/V，避免通用路径的页面拼装；load-and-reduce 加速行最大值计算；causal softmax 被拆成无 mask 前缀和有 mask 后缀，并在行最大值不变时跳过不必要的校正缩放；最终用合并写回和提前释放输出资源收紧 epilogue。这些改动联合将专家已深度优化的单 Kernel 路径再加速 1.17×。这说明AKA 不仅搜索 tile，而是可以联合重构跨 tile 调度、数据搬运、softmax 和写回流水。

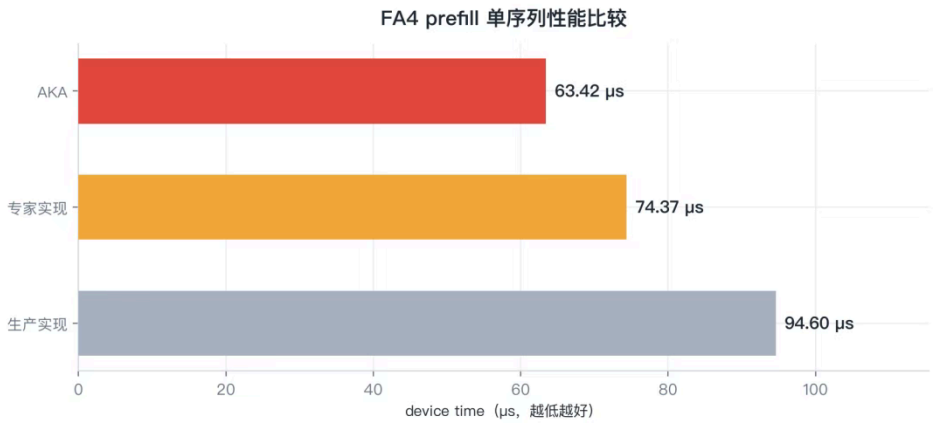


图 4 | FA4 prefill 常规单序列性能比较。

3.2 FA4 decode：原生适配 page128，q_len = 4 MTP 专项调度

在 q_len = 4 MTP 的 10 个 shape 上，AKA 全部领先专家，几何平均约 1.19×；相对生产实现约 2.15×。

Qwen3.8-Max 的 decode 同时包含 q_len=1 单 token decode 和 q_len = 4 MTP 两种负载。生产页表采用 page128，而上游通用入口主要围绕 page64 设计，接入时通常需要由适配层将 KV 数据零拷贝重解释为两个 page64，并同步重展开页表。AKA 让 Kernel 原生接受 `table_page_size=128`，直接匹配生产布局，并把 q_len=1 与 q_len = 4 作为两种不同的调度问题处理。

q_len = 4 路径采用 `prediction_tile=2`、`sequence_tile=128`、`cluster_kv=2` 等组合，配合 split/reduction 搜索；编译期 mask 特化、循环剥离和静

态 trip count 则消除了通用路径中的边界分支。q_len=1 路径与专家基本持平，整体优势来自最难的 q_len = 4 MTP 段。完整 20-shape 结果为相对生产 1.55×、相对专家 1.075×。

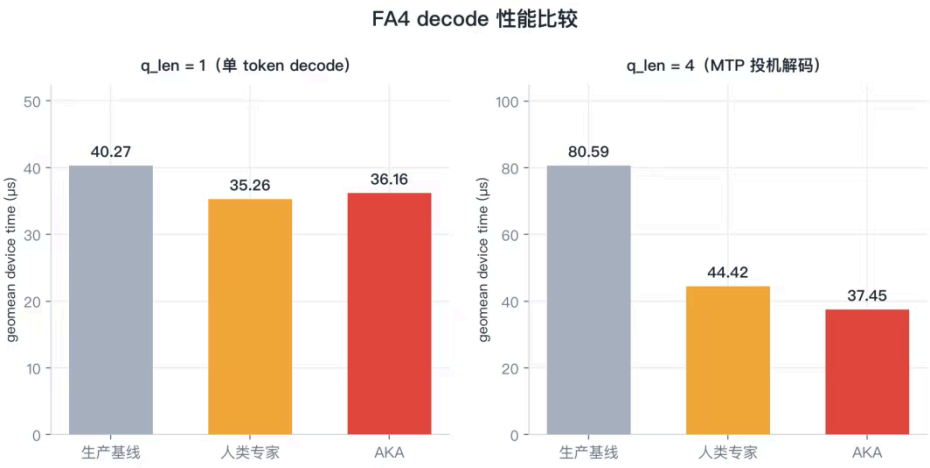


图 5 | FA4 decode 在 q_len = 1 与 q_len = 4 MTP 两类负载上的性能比较。

3.3 GDN prefill：AKA 发现新的流水设计

10 个 shape 的几何平均时延为：AKA 179.830 μs，专家 192.176 μs，FlashInfer core 229.604 μs。

这条线路从公开 FlashInfer chunked 主干启动，搜索期间没有向 Agent 提供专家 M64 源码或策略；最终，AKA 在公开主干上独立走出另一条路径。GDN（Gated DeltaNet）是 Qwen 混合架构中的线性注意力算子，也是本次三条线路中自主搜索周期最长的一条。

两条 M64 路径的核心差异，是并行问题和标量流水都被重新设计。专家用 `cluster_shape_mnk=(1,2,1)` 由两个 CTA 协作拆分 value；AKA 保持 `cluster_shape_mnk=(1,1,1)`，在问题描述层面把一个物理 value head 重述为两组行块，让调度器直接分配。在 B=1、M64 的稳定路径上，专家采用 g2r 与 chunk lookahead；AKA 则增加 scalar-TMA warp，将四个 head 的标量列合并搬入共享内存，并配合静态 TMA 描述符与 q3/k4/v2/o1 流水。

在这一过程中，NCU 与内核级时间线 profiler（IKeT）的联合反馈定位了标量搬运和同步等待问题，推动了 scalar-TMA warp、事件流与流水阶段的调整。

为了找到这一版，AKA 进行了 117 轮有效探索。AKA 为 179.830 μs，低于专家 192.176 μs 和 FlashInfer core 229.604 μs。

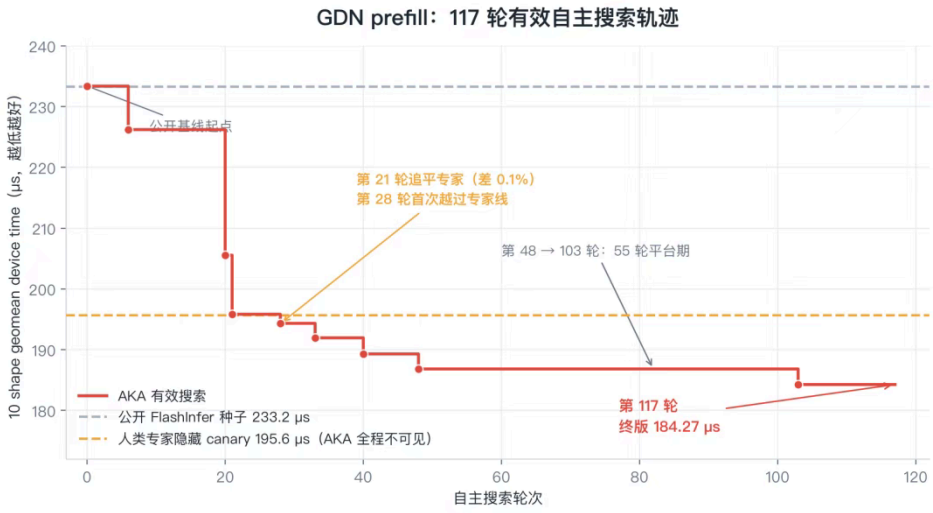


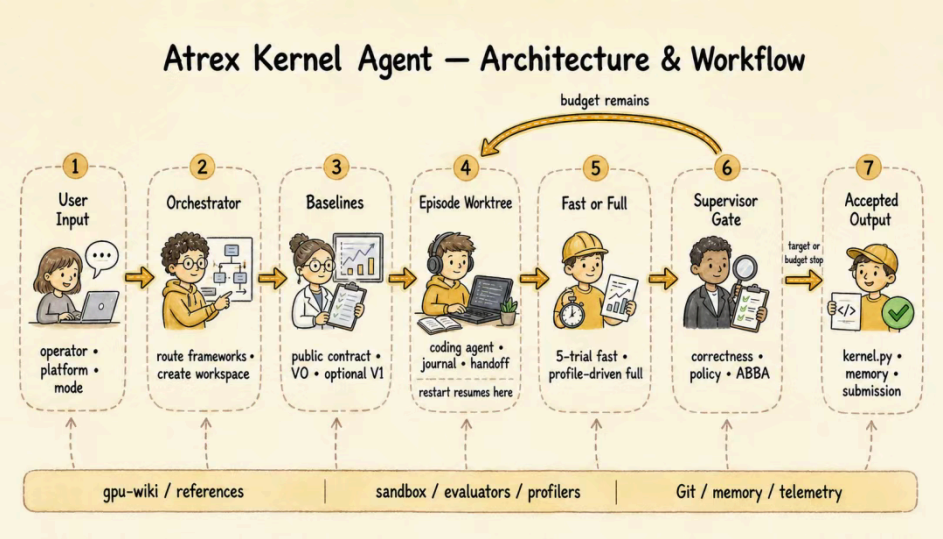
图 6 | GDN prefill 的 117 轮自主搜索轨迹

三条线呈现出同一条路径：专家把既定接口内的空间调到极致，AKA 则把接口边界、Kernel 数量、计算分解和调度结构一起纳入搜索。FA4 prefill 的 batch-1 persistent grid、decode 的 q1/q4 分治和 GDN 的 value 行块拆分，都是 Plan 先形成结构假设，再由 Agent 经过编译、正确性和完整 workload 性能验证得到的结果。专家优化“这个接口下怎么最快”，AKA 进一步搜索“这个接口本身该长什么样”。

04
AKA 的系统设计

AKA 是一套持续运行的 GPU 性能工程闭环。首先通过 Atrex-Bench 将真实业务 workload 转化为算子契约和评测基线；Plan 结合 GPU Wiki、计算模式及硬件特性提出数学变换、数据布局和流水线设计；编码 Agent 负责修改 Kernel 与运行时路由；Agate 提供隔离、可复现的 GPU 执行和评测环境；NCU 与内核级时间线 profiler 提供性能反馈；Supervisor 则机械化管理正确性门禁、完整 workload 复测、版本管理和失败回退。

```
真实业务 workload
↓
Atrex-Bench 契约与性能基线
↓
Plan + GPU Wiki 提出结构优化方案
↓
Agent 修改 Kernel 与运行时路由
↓
Agate：编译 → 正确性验证 → 全 shape 性能复测
↓
NCU / IKeT 反馈性能瓶颈
↓
Supervisor 晋级、回退并沉淀经验
└———→ 下一轮搜索
```



4.1 目标：Atrex-Bench 让真实 workload 成为优化对象

Atrex-Bench 的算子接口和 shape 来自百炼平台的真实部署 workload。Agent 面对完整 shape 集合、生产页表和既定 buffer 契约，而不是一个容易过拟合的示例输入。它不仅负责计时，也定义优化目标：真实流量中最昂贵、最值得优化的算子，获得更高的探索优先级。

4.2 设计：Plan 在写代码之前完成结构推演

Plan 模块结合算子的接口范围、计算模式和硬件特性，先建立可搜索的设计空间，再让 Agent 编码实现。设计空间包括数学等价变换、tile 与 layout 重组、pipeline 和 warp role 设计，以及主机端与设备端边界的移动。

4.3 反馈：从 NCU 指标到 Kernel 内部时间线

NCU 可以告诉 Agent 吞吐、occupancy、访存和 warp stall 的宏观状态；在最新架构下，TMA、MMA、barrier 和 warp 角色之间的细粒度耦合，往往决定最后几个百分点。AKA 在工作流中引入内核级时间线 profiler（如IKET），记录 warp role、pipeline wait、TMA/MMA 与 barrier 的内部事件。

NCU 回答“哪里慢”，细粒度profiler可以进一步回答“哪个 warp 在什么同步点等待”。前述 GDN 案例中 scalar-TMA warp、事件流和流水阶段的改动，正是由这类反馈提出并验证的结构假设。

4.4 验证：稳定更优的版本才会晋级

Supervisor 对预算、正确性、测量、晋级和回退负责。候选必须先通过正确性硬门，再接受完整 workload 评测和独立复测；错误或回归版本自动回退，稳定更优的版本才会晋级。代码、Profile、知识记忆和实验日志同步保留，使每个版本都可以复现、解释和回滚。

05
Harness 自进化：把优化流程也纳入实验

AKA 正在实验性地把进化对象从 Kernel 扩展到承载优化流程的 harness。系统汇总优化轨迹，从历次实验中提炼高价值上下文、有效 profiler 信号和成功操作序列，用于改进提示词结构、知识路由、预算分配和复测策略。

进化前的 harness 在第 28 轮越过专家线，在第 48 轮达到 186.799 μs，再经过 55 轮探索于第 103 轮突破到 184.268 μs。进化后 harness 的另一次 GDN 实验保留了 23 个已完成 Attempt，并行运行两条 Trajectory。按 Excel 中“最终 Kernel”的权威延迟计算累计最优值，第 11 个完成 Attempt 已越过专家线，第 22 个达到 187.299 μs。从 Token 成本看，进化后 harness 约在 1.56 亿 tokens 越过专家线，约在 3.08 亿 tokens 达到 187.299 μs；进化前轨迹越过专家线用了约 7.30 亿已记录 tokens。这表明新 harness 更早进入高性能区间；

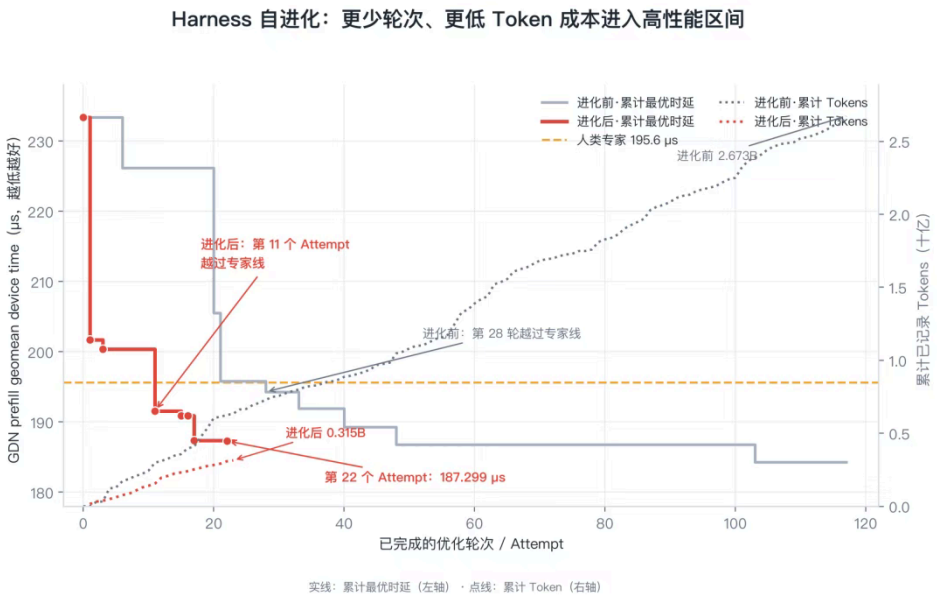


图 7 | 进化前后 harness 在 GDN prefill 上的合并轨迹。横轴统一为完成轮次：左轴实线表示累计最优时延，右轴点线表示累计 Token 用量。

Harness 自进化的目标，是让同样的 token 和 GPU 预算覆盖更多高价值假设，并更早进入有效搜索区间。AKA 不只优化算子，也开始优化寻找算子的流程。

06
结语：从“会优化”到“能交付”

Kernel Agent 的公开研究已经表明，Agent 可以在标准题目上生成并优化 GPU Kernel；AKA 这次把问题推进到了生产契约：面对 Qwen3.8-Max 的真实关键算子、复杂调度约束，以及专家已经深度优化过的实现，Agent 仍然找到了新的执行路径，并在三条独立算子线的几何平均时延上全部低于专家版本。

AKA 将反馈、设计和生产验证组织成一个可以持续运行的闭环，把专家经验之外的结构维度纳入搜索，也把一次性能突破沉淀为下一次优化的系统能力。这是 Kernel Agent 从公开评测走向生产算子的一次重要跨越。后续，这套闭环将继续扩展到更多 Qwen 生产算子、融合路径和硬件形态，让“生产 workload → 自动优化 → 可复现验证 → 工程集成”成为 GPU 软件栈的常规开发路径。

开源项目与快速体验

AKA 支持 NVIDIA、AMD 和 PPU 等硬件平台，适配 Claude、Qoder、Codex、Pi 等编码 Agent，并覆盖 Triton、CuTe DSL、CUDA、FlyDSL、TileLang 等编程范式。无论是公开 Benchmark 还是 Atrex-Bench 真实生产任务，AKA 都用同一套可复现、可审计、可恢复的工程流程管理优化过程。

AKA 已正式开源：GitHub | https://github.com/alibaba/atrex-kernel-agent。克隆仓库后，可以在仓库根目录下启动 Claude Code、Codex 或 Qoder，直接描述目标算子、硬件平台和优化预算，由 Agent 调用 AKA 的统一编排入口启动优化任务；完整环境准备、命令示例和产物说明请参考仓库中的 Quick Start。同时，阿里巴巴 TRE AI 系统工程团队开源其生产级异构算子库：GitHub | https://github.com/alibaba/atrex-kernels。

如果你正在研发推理引擎、训练框架、GPU 软件栈或高性能算子，欢迎 Star、试用并通过 GitHub Issues 反馈问题；也欢迎共同贡献新的算子任务、硬件后端、编程范式与 GPU 优化知识。



欢迎留言一起参与讨论~

AI Native · 目录 ≡

<上一篇
代码生成越来越快，软件交付真正的瓶颈转向了哪里？

下一篇 >
AI Native 回忆录：稳定性前端 S1 实践复盘

阅读 1753

留言 1

写留言



3ბ◯3ბ◯安然 云南 9月28日

博博好棒👍💗

首评